

Tugas 3 Image Processing
“Implementasi Konvolusi Citra Menggunakan Linear Filter 5×5 ”



NAMA : FIKHRI HANIF
NIM : 2311533007

DOSEN PENGAMPU :
AFDHAL DINILHAK, M. Kom.

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG

A. PENDAHULUAN

Pengolahan citra digital merupakan bidang dalam ilmu komputer yang berfokus pada pemrosesan dan analisis citra digital menggunakan algoritma komputer. Salah satu teknik yang sering digunakan dalam pengolahan citra adalah image filtering.

Filtering merupakan proses memodifikasi nilai pixel pada suatu citra berdasarkan nilai pixel tetangganya. Salah satu metode filtering yang umum digunakan adalah konvolusi (convolution).

Konvolusi bekerja dengan cara mengalikan setiap nilai pixel pada citra dengan nilai pada kernel kemudian menjumlahkannya untuk menghasilkan nilai pixel baru. Secara matematis, operasi konvolusi dapat dituliskan sebagai:

$$G(x, y) = \sum_{i=-k}^k \sum_{j=-k}^k H(i, j) \cdot F(x - i, y - j)$$

Keterangan:

- $F(x, y)$ adalah citra input
- $H(i, j)$ adalah kernel atau filter
- $G(x, y)$ adalah citra hasil konvolusi

Pada praktikum ini digunakan kernel rata-rata (mean filter) berukuran 5×5 yang berfungsi untuk melakukan smoothing atau blur pada citra.

Kernel yang digunakan adalah:

$$K = \frac{1}{25} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

Kernel tersebut akan menghitung rata-rata dari 25 pixel tetangga sehingga citra menjadi lebih halus.

Dalam implementasi konvolusi terdapat permasalahan pada pixel tepi (border pixel) karena kernel tidak dapat sepenuhnya menutupi bagian tepi citra. Oleh karena itu digunakan tiga pendekatan, yaitu:

1. Mengabaikan pixel pinggir
2. Menambahkan padding pada citra
3. Duplikasi pixel tepi

B. TOOLS YANG DIGUNAKAN

1. Python
2. Google Colab
3. Library dasar:
 - Numpy
 - PIL (untuk membaca gambar)

C. LANGKAH-LANGKAH IMPLEMENTASI

1. Import Library yang akan digunakan

```
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
```

Penjelasan:

- numpy digunakan untuk operasi matriks
- PIL digunakan untuk membaca dan menyimpan citra
- matplotlib digunakan untuk menampilkan citra

2. Membaca Gambar

```
img = Image.open("gambar.jpg").convert("L")
img_array = np.array(img)

plt.imshow(img_array, cmap='gray')
plt.title("Citra Asli")
plt.axis('off')
```

Pada tahap ini citra dibaca dari file yang telah diupload ke Google Colab, kemudian citra diubah menjadi grayscale.

Penjelasan kode:

- Image.open() digunakan untuk membuka citra
- .convert("L") digunakan untuk mengubah citra menjadi grayscale
- np.array() mengubah citra menjadi array matriks
- plt.imshow() digunakan untuk menampilkan citra

Output:



3. Membuat Kernel Rataan 5×5

```
kernel = np.ones((5,5)) / 25  
print(kernel)
```

Kernel dibuat menggunakan array numpy.

Penjelasan kode:

- Kernel rataan 5×5 memiliki nilai 1 pada setiap elemen yang kemudian dibagi dengan 25 sehingga menghasilkan nilai rata-rata dari 25 pixel.
- Kernel ini berfungsi untuk melakukan smoothing atau blur pada citra.

Output:

```
[[0.04 0.04 0.04 0.04 0.04]  
 [0.04 0.04 0.04 0.04 0.04]  
 [0.04 0.04 0.04 0.04 0.04]  
 [0.04 0.04 0.04 0.04 0.04]  
 [0.04 0.04 0.04 0.04 0.04]]
```

4. Implementasi Konvolusi

1) Metode 1 - Mengabaikan Pixel Pinggir

```
h, w = img_array.shape  
output1 = img_array.copy()  
  
for i in range(2, h-2):  
    for j in range(2, w-2):  
  
        total = 0  
  
        for m in range(-2,3):  
            for n in range(-2,3):  
  
                total += img_array[i+m][j+n] * kernel[m+2][n+2]  
  
        output1[i][j] = total
```

Pada metode ini pixel di bagian tepi tidak diproses sehingga nilainya tetap sama dengan citra asli.

Penjelasan kode:

- Loop hanya berjalan dari 2 sampai h-2
- Hal ini dilakukan agar kernel tidak keluar dari batas citra
- Pixel tepi tidak berubah

2) Metode 2 - Zero Padding

```
pad = 2
```

```

padded = np.pad(img_array, pad, mode='constant',
constant_values=0)

output2 = np.zeros_like(img_array)

for i in range(img_array.shape[0]):
    for j in range(img_array.shape[1]):

        total = 0

        for m in range(5):
            for n in range(5):

                total += padded[i+m][j+n] * kernel[m][n]

        output2[i][j] = total

```

Pada metode ini ditambahkan **padding bernilai 0** di sekitar citra.

Penjelasan kode:

- Padding ditambahkan dengan nilai 0
- Ukuran padding adalah 2 pixel karena kernel 5×5
- Kernel dapat diproses hingga ke bagian tepi citra

3) Metode 3 - Duplikasi Pixel Tepi

```

padded2 = np.pad(img_array, 2, mode='edge')

output3 = np.zeros_like(img_array)

for i in range(img_array.shape[0]):
    for j in range(img_array.shape[1]):

        total = 0

        for m in range(5):
            for n in range(5):

                total += padded2[i+m][j+n] * kernel[m][n]

        output3[i][j] = total

```

Metode ini menduplikasi nilai pixel di tepi citra.

Penjelasan kode:

- Mode edge akan menyalin nilai pixel tepi sehingga padding memiliki nilai yang sama dengan pixel terdekat.

5. Menampilkan Hasil

```
plt.figure(figsize=(12,6))

plt.subplot(1,4,1)
plt.imshow(img_array, cmap='gray')
plt.title("Original")

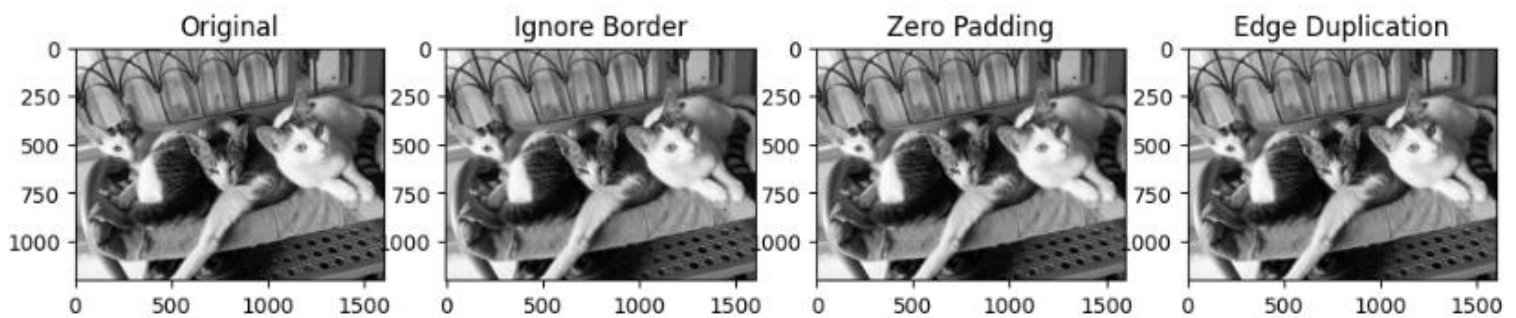
plt.subplot(1,4,2)
plt.imshow(output1, cmap='gray')
plt.title("Ignore Border")

plt.subplot(1,4,3)
plt.imshow(output2, cmap='gray')
plt.title("Zero Padding")

plt.subplot(1,4,4)
plt.imshow(output3, cmap='gray')
plt.title("Edge Duplication")

plt.show()
```

Output:



D. HASIL DAN ANALISIS

Berdasarkan hasil percobaan:

1. Metode Ignore Border
 - Pixel tepi tidak berubah
 - Efek smoothing hanya terjadi di bagian tengah citra
2. Zero Padding
 - Pixel tepi diproses dengan tambahan nilai 0
 - Kadang menghasilkan sedikit artefak gelap di tepi citra

3. Edge Duplication
 - Pixel tepi diduplikasi
 - Hasil smoothing terlihat lebih natural dibanding zero padding

E. KESIMPULAN

1. Operasi konvolusi dapat digunakan untuk melakukan smoothing pada citra menggunakan kernel rata-rata 5×5 .
2. Implementasi konvolusi dapat dilakukan secara manual menggunakan perulangan tanpa menggunakan library pengolahan citra seperti OpenCV.
3. Penanganan pixel tepi dapat dilakukan dengan beberapa metode yaitu mengabaikan pixel pinggir, menggunakan zero padding, dan duplikasi pixel tepi.
4. Metode duplikasi pixel tepi memberikan hasil yang lebih stabil dibandingkan zero padding.

LINK GITHUB :

https://github.com/fikhrihanif/2311533007_FikhriHanif_ImageProcessing/tree/main/Tugas3_ImageProcessing